CE205: Databases and Information Retrieval

Lab Week 11

NoSQL: MongoDB

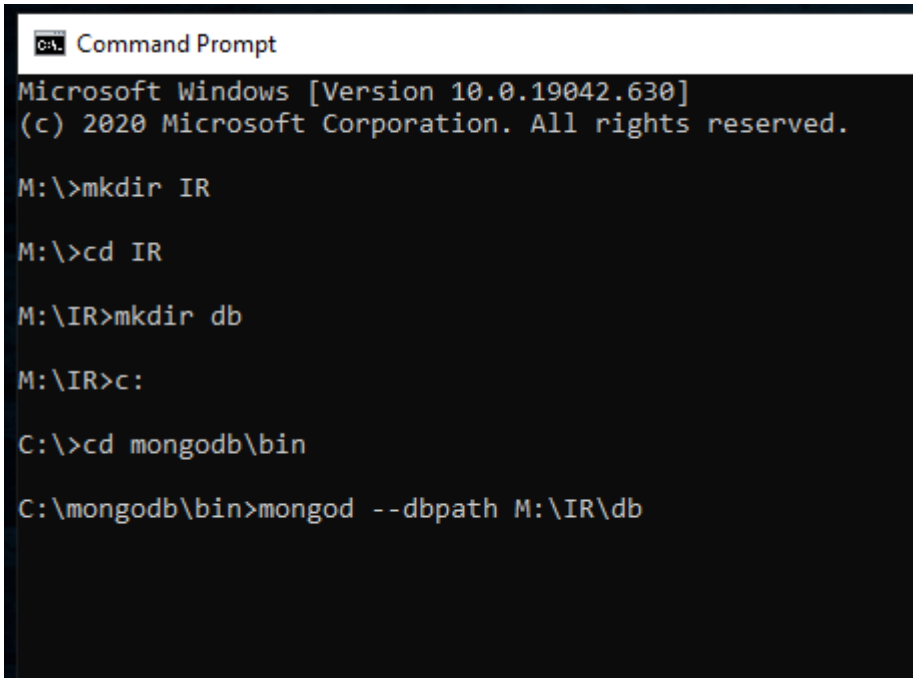
Objective

The aim is to try out MongoDB, working with the Python client interface, to gain an understanding of how it works.

Instructions

Start MongoDB Server:

Open CMD and do the following:



- It will create directories on M drive
- The command mongod --dbpath M:\IR\db (take care of double dashes before dbpath) will start mongo database server.

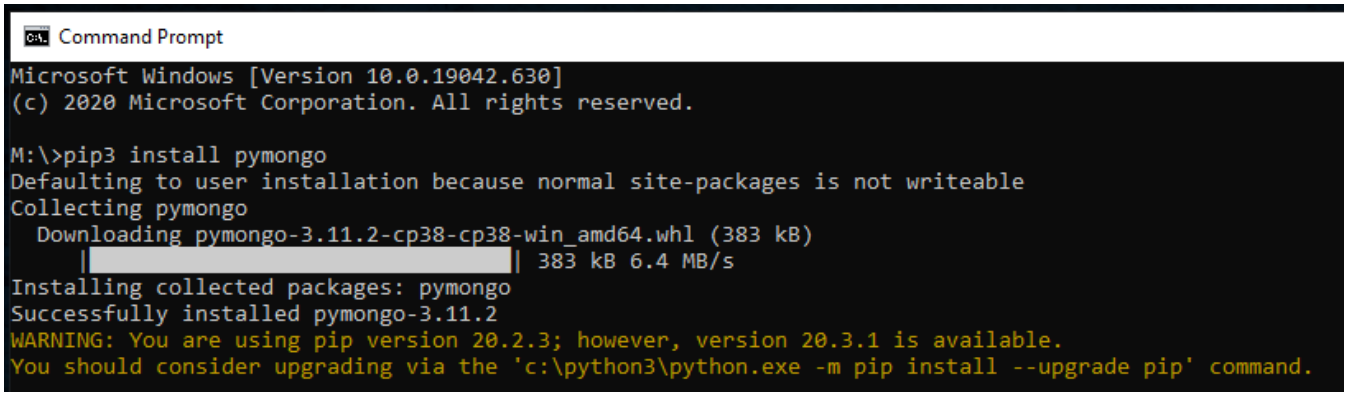
Do not close the command prompt (leave the server running).

Install Library 'pymongo':

Open second CMD, install the library pymongo of Python (we need it later) using the following command:

pip3 install pymongo

You will have the following screen



Ignore the warning.

Sample Dataset:

Now, get the sample collection of documents (primer-dataset.json) and save it as ***M:\IR\db\primer-dataset.json***.

Get file from

https://raw.githubusercontent.com/mongodb/docs-assets/primer-dataset/primer-dataset.json

or

from Moodle under Unit 9 (Lab Week 11).

A sample document of the collection is shown in the appendix.

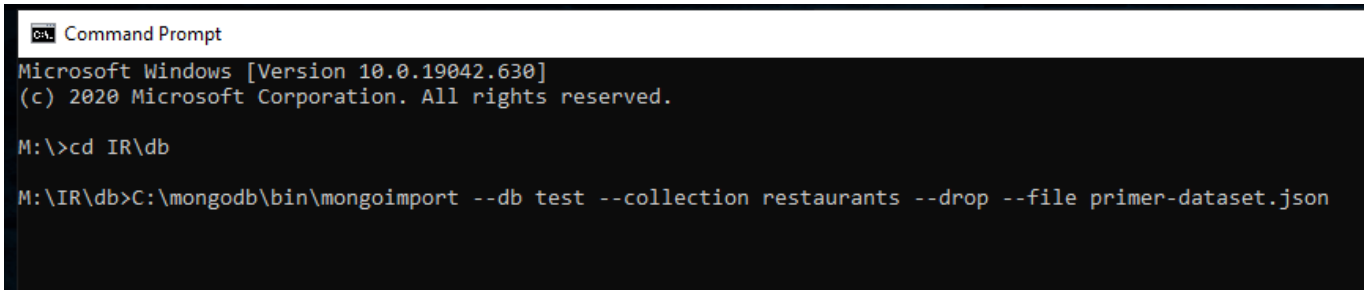
Load Dataset:

Go to ***M:\IR\db*** and write the command

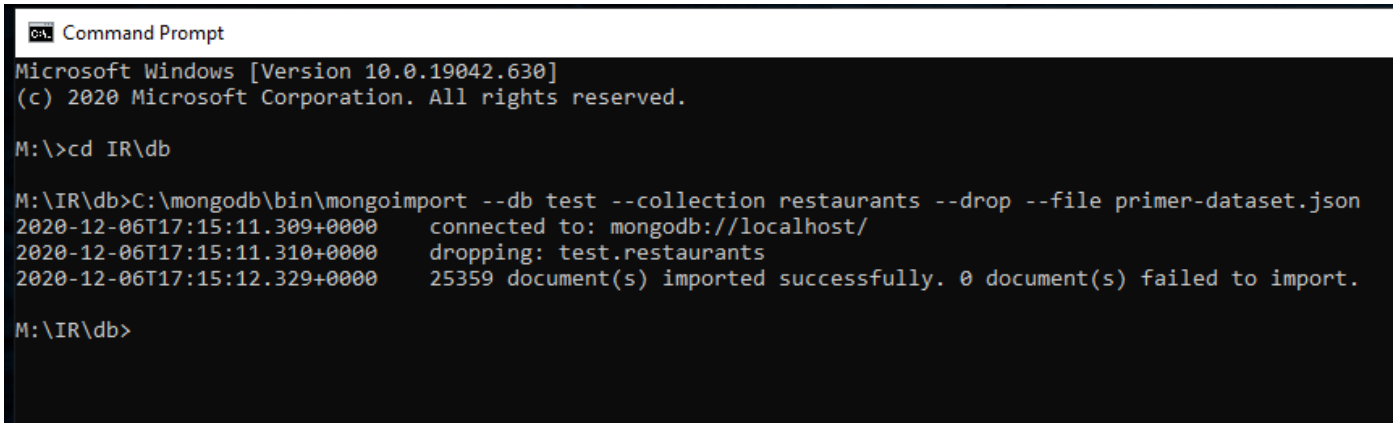
C:\mongodb\bin\mongoimport --db test --collection restaurants --drop --file primer-dataset.json

*(**drop** obtains an exclusive lock on the specified collection for the duration of the operation. All subsequent operations on the collection must wait until drop releases the lock.)*

It will look like



After executing the command, you will have



It will create a database called test and a collection called restaurants within the test database.

Start Python:

Now, start Python by typing python in the same CMD, and follow these steps.
(Lines beginning with # are comments for your information - do not type those lines)

(see slides 2 to 4)

```
from pymongo import MongoClient
client = MongoClient()
db = client.test
# Comment: This is the name of our database containing primer-dataset.json
```

Using Python, get records through queries:

(see slides 5 to 6)

```
# Find all documents in the collection:
cursor = db.restaurants.find()
for document in cursor:
    print(document)
```

*(careful **p** (of print) should be under **d** (of document))*

```
# NB press RETURN twice after the print line to make it work.
# prints out every document, a vast amount of data but does not take too long...
# wait patiently, it will finish...
# scroll back and have a look at some of the documents to see the structure

# Now, find documents whose borough field equals "Manhattan":
cursor = db.restaurants.find({"borough": "Manhattan"})
for document in cursor:
    print(document)
# still a vast number...
# wait patiently...
# verify that the last few are indeed all in Manhattan by scrolling back.
```

You can also use count( ) and slice with any other command

```
#If you want to see, how many documents are returned
cursor = db.restaurants.find({"grades.grade": "B"}).count()
print(cursor)
#If you want to get first 5 (for example) documents
cursor = db.restaurants.find({"borough": "Manhattan"})
slice=cursor[1:6];
for document in slice:
    print(document)
```

```
# Now, find documents where zipcode within address field has value 10075:
cursor = db.restaurants.find({"address.zipcode": "10075"})
for document in cursor:
    print(document)
# verify that they are all in 10075.

# Now, find documents where field grade in an array grades has value "B":
cursor = db.restaurants.find({"grades.grade": "B"})
for document in cursor:
    print(document)
# a lot once again...
# verify last few.

# Now, find documents whose grades array contains an embedded document with a
# field "score" greater than 30:
cursor = db.restaurants.find({"grades.score": {"$gt": 30}})
for document in cursor:
   print(document)
# quite a lot.
# verify some of them.

# Find documents whose grades array contains an embedded document with a
# field "score" less than 10:
cursor = db.restaurants.find({"grades.score": {"$lt": 10}})
for document in cursor:
    print(document)
# Verify.

# logical AND (two conditions):
cursor = db.restaurants.find({"cuisine": "Italian", "address.zipcode":"10075"})
for document in cursor:
    print(document)
```

```
# verify.

# logical AND (three conditions):
cursor = db.restaurants.find({"cuisine": "Italian", "address.zipcode": "10075",
"address.building": "1471"})
for document in cursor:
    print(document)
# verify.

# logical OR:
cursor = db.restaurants.find( {"$or": [{"cuisine": "Italian"},{"address.zipcode": "10075"}]})
for document in cursor:
    print(document)
# verify.
# two ANDs within an OR:
cursor = db.restaurants.find( {"$or": [{"cuisine": "Italian", "address.building": "101-16"},
{"address.zipcode": "10075","address.building":"1501"}]})
for document in cursor:
    print(document)
# verify.

# sort results:
import pymongo
cursor = db.restaurants.find({"cuisine": "Bakery"}).sort([
("borough", pymongo.ASCENDING),
("address.zipcode", pymongo.DESCENDING)
])
for document in cursor:
    print(document)
# verify that this works and does order the output in this way.
```

APPENDIX: A Sample Document from the Collection Restaurant

The collection Restaurant contains account details for restaurants. Each document contains details on the restaurant such as its address, borough, review scores, its name, and the type of food it serves.


```
{
 "address": {
    "building": "8825",
    "coord": [-73.8803827, 40.7643124],
    "street": "Astoria Boulevard",
    "zipcode": "11369"
 },
 "borough": "Queens",
 "cuisine": "American",
 "grades": [ {
    "date": {"$date": "2014-11-15T00:00:00.000Z"},
    "grade": "Z",
    "score": 38
 },
 {
    "date": {"$date": "2014-05-02T00:00:00.000Z"},
    "grade": "A",
    "score": 10
 },
 {
    "date": {"$date": "2013-03-02T00:00:00.000Z"},
    "grade": "A",
    "score": 7
 },
 {
    "date": {"$date": "2012-02-10T00:00:00.000Z"},
    "grade": "A",
    "score": 13
 }],
    "name": "Brunos On The Boulevard",
    "restaurant_id": "40356151"
}
```